

Gradient Ascent

A Basic Optimization Algorithm

At a Glance

Applicability: Continuous Optimization Problems

Abstract Description: Start at one point, find a direction to jump in based on the growth of the function, repeat.

Parameters:

- *starting point*: which point to start on
- *learning_rate*: how far to jump relative to the growth

The Algorithm

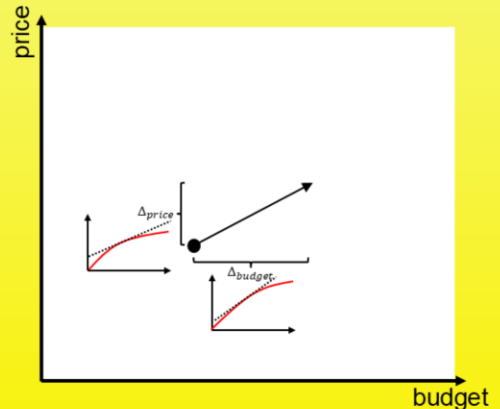
1) Start at a point and calculate the gradient

- The gradient is component-wise defined by the gradients of all parameters
- Gradient of 1 parameter = first derivation
- Gradient approximation via close by point

2) Move into the direction of the gradient

- Distance is influenced by learning rate
- If the resulting point is worse than the current one we are finished.

3) Repeat from 1.



The algorithm works a lot like hill climbing but doesn't need to rely on a neighbourhood relationship (which isn't present in continuous search spaces). Instead, it uses the gradient. The gradient is a number that tells us how much the function grows if we make a step into a particular direction. Ideally, we can get this gradient based on the first derivation of the function. Often, that isn't possible. In this case we can approximate the gradient by making a small step into the direction and checking how much our function changes.

In more detail, this is how the algorithm works as follows:

- 1) Start from the current point and calculate the gradient. We calculate gradients one parameter at a time. In our example, we are calculating two gradients Δ_{price} and Δ_{budget} by fixing the other parameter and calculating the gradient of the other.
- 2) Move into the direction of the gradient. This means taking the current point and adding the gradients for all parameters to it. This results in the vector in the example illustration in the slide. Note that often we want to regulate how far we jump. For this purpose, we have a learning rate that we can choose to adjust our step size. If the resulting point is smaller – or very close – to our current point, then we can stop the algorithm.
- 3) If we haven't stopped the algorithm the calculated point becomes our new start point and we repeat from step 1.